

# CS 5542 Challenge 1: Big Data Job Search Application

Student: Vince Nguyen

GitHub: <https://github.com/VinceNguyen000/2026-agentic-ai-job-search>

Submission: September 11, 2026

## 1. Problem and Big Data Goal

I designed this application for job seekers who spend too much time manually comparing resumes, career goals, and job postings. A seeker provides career goals, industries, experience, education, skills, certifications, location, salary, work mode, work type, and availability. A job record provides its title, description, responsibilities, required and preferred skills, experience, education, location, salary, work mode, work type, and deadline.

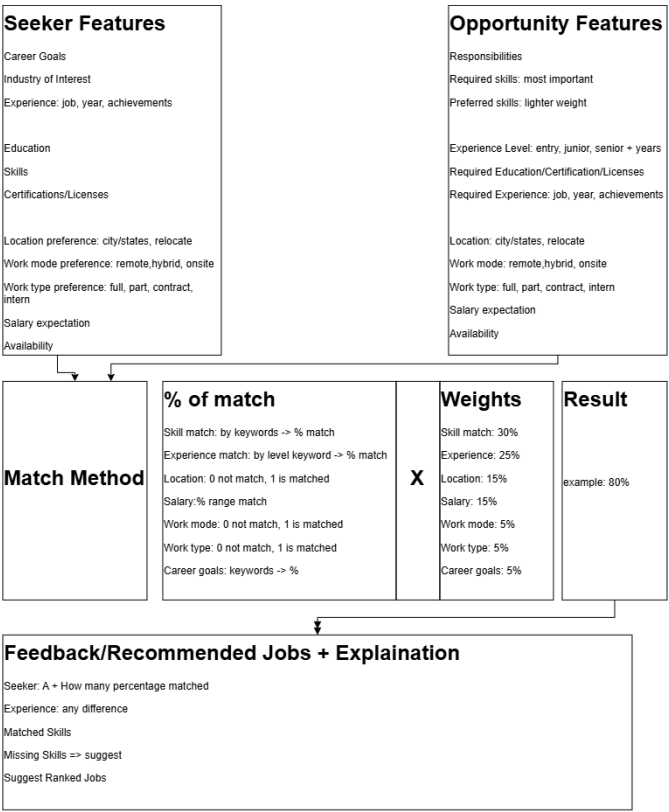
The system returns a match percentage, score breakdown, matched and missing skills, an explanation, ranked jobs, a tailored application draft, and an upskilling roadmap. A successful match is realistic and explainable rather than based only on a shared job title.

The problem has the five big-data characteristics: high Volume of listings and resumes, high Velocity as postings change, Variety in structured and unstructured fields, Veracity problems from missing or inconsistent data, and Value from personalized recommendations.

## 2. Human Design

Before using AI, I designed a sequential workflow: seeker features and opportunity features feed a matching method; each criterion produces a percentage; weights produce a final result; and the application returns feedback and recommended jobs. The weights were skills 30%, experience 25%, location 15%, salary 15%, work mode 5%, work type 5%, and career goals 5%.

My original design used explicit structured attributes, keyword overlap, experience-level comparison, salary ranges, and binary checks for strict preferences. Its strengths were transparency and simplicity. Its limitations were rigid location rules, direct keyword dependence, and no semantic understanding or scalable ingestion pipeline.



## 3. AI Design

I asked Antigravity to implement Python data classes for Seeker, Opportunity, and MatchResult, a seven-factor matcher, and an autonomous job-search agent. The AI proposed modular files for models, matching, utilities, agent tools, and the CLI. It generated tools for profile inspection, job search, fit evaluation, skill-gap analysis, application drafting, and an upskilling roadmap.

AI improved implementation speed, modularity, documentation, and the transition from a static formula to an actionable assistant. It initially misunderstood two cases. A Jaccard union denominator made short career goals appear unrelated to long job descriptions. A minimum token length removed important terms such as AI, ML, and JS. The generated code was executable, but human inspection was necessary because executable code can still produce misleading results. The local JSON design was suitable for an MVP but not for millions of live records.

## 4. Human-AI Co-Design

I kept the original problem, seven criteria, fixed weights, explainable results, and skill-gap feedback. I accepted AI-generated data classes, modularization, aliases, the agent workflow, application drafting, and roadmap generation. I rejected the misleading career-goal denominator and corrected token filtering so short technical acronyms remain available.

I then added a reusable ingestion layer, missing-value handling, duplicate checks, BM25 retrieval, local dense-vector retrieval, hybrid RAG evidence, descriptive analytics, evaluation artifacts, and a Streamlit interface. Human judgment controlled the weights, validated output quality, and separated implemented features from future ideas. External embedding APIs, pretrained semantic models, Spark, live APIs, streaming, cloud processing, and vector databases are not implemented in this version.

## 5. Data Collection, Storage, and Processing

Dataset: Agentic Job Search Demonstration Dataset.

Source: Author-created curated demonstration data included in the repository.

URL status: No external source URL; the repository files are the reproducible source.

Current size: 4 seeker profiles, 6 job opportunities, and 26 required-skill entries.

The data is stored as JSON in examples/seekers.json and examples/opportunities.json. src/data\_pipeline.py converts records into typed dataclasses and controlled enum values. Missing optional lists become empty lists; missing salary information is treated as unknown; required identifiers are checked; and duplicate seeker names or job IDs are reported.

Text is lowercased, punctuation is removed, stop words are filtered, and technical aliases are recognized. Required and preferred skills are kept separate. Personal career evidence is rebuilt into 13 clean local chunks from the resume, GitHub project notes, and preferences files. JSON and in-memory Python processing are appropriate for this reproducible MVP. Production use would require external APIs or database/cloud storage, stronger duplicate resolution, and distributed processing.

## 6. Analytics and Matching Method

The final score is:

Skills 30% + Experience 25% + Location 15% + Salary 15%  
+ Work mode 5% + Work type 5% + Career goals 5%

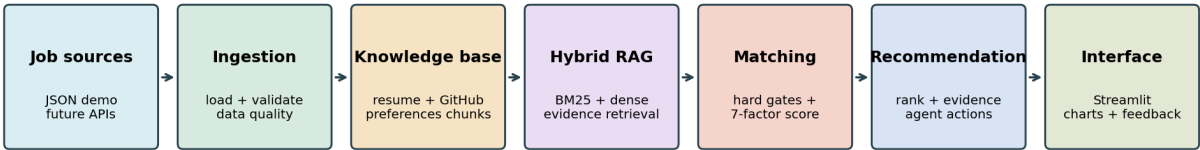
Skills compare required and preferred qualifications. Experience uses an entry-to-executive hierarchy. Location supports exact and relocation matches. Salary uses range overlap. Work mode and work type use controlled preferences. Career goals use meaningful keyword containment over job titles, descriptions, and responsibilities.

BM25 in src/retrieval.py retrieves lexical candidates from job titles, descriptions, responsibilities, and skills. src/rag.py builds a personal career knowledge base and combines BM25 with deterministic 256-dimensional hashed dense vectors using cosine similarity. A hard-gating step checks salary and work-mode prerequisites before soft scoring. Retrieved chunks include chunk IDs, titles, source metadata, sparse scores, dense scores, and a combined hybrid score. The agent uses these chunks as evidence context for job-specific analysis. This is a local hybrid RAG implementation; it does not call an external embedding API or LLM at runtime. Antigravity assisted development but is not an external LLM service at runtime.

## 7. Analytics and Visualization

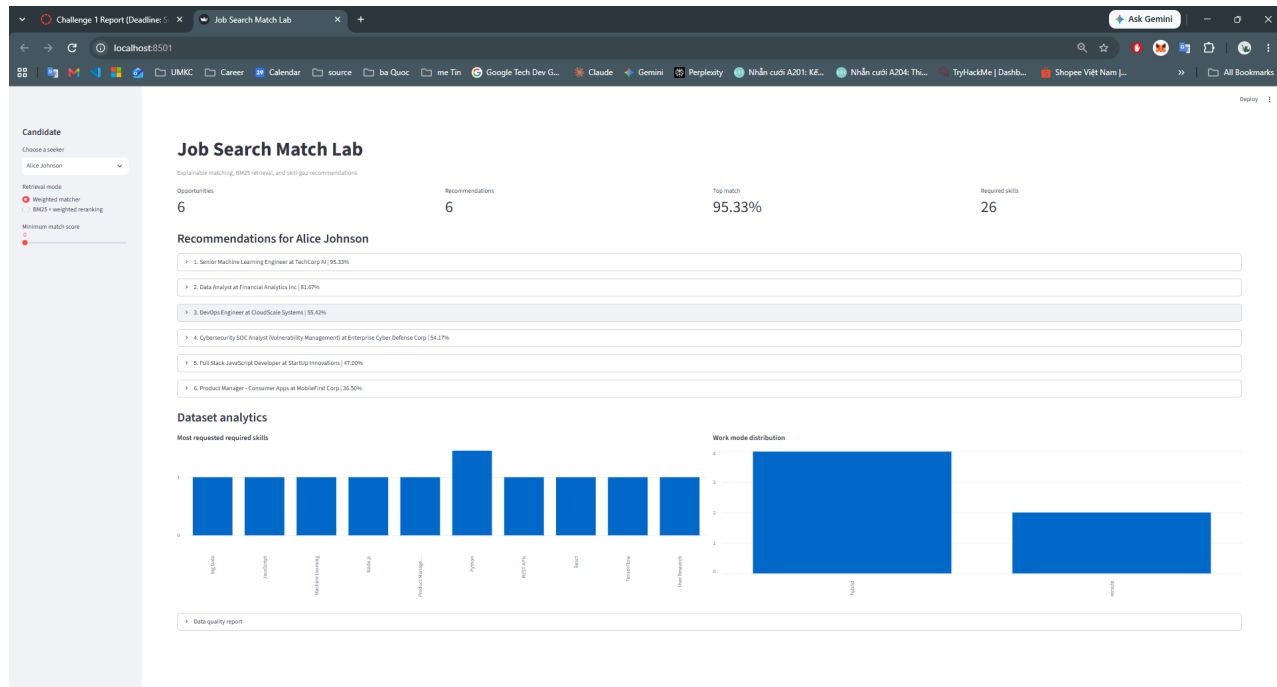
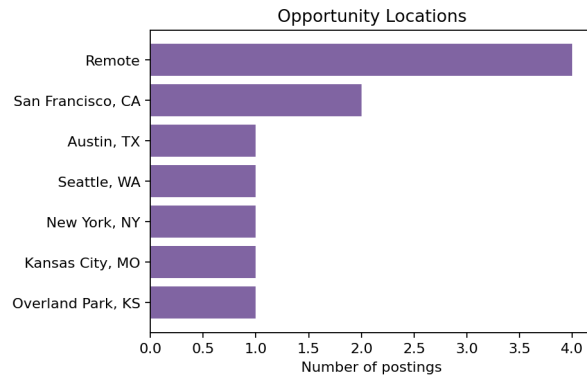
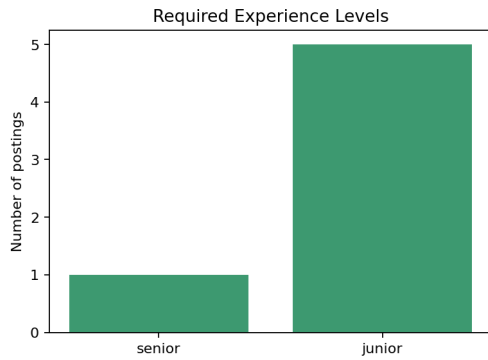
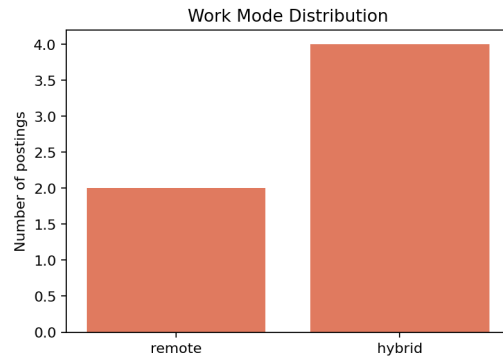
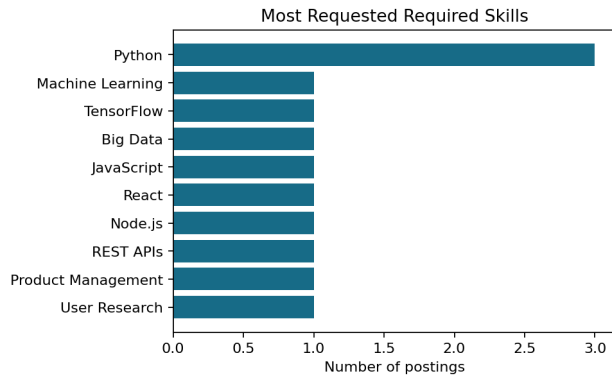
The analytics pipeline reports skill frequency, job titles, locations, work modes, experience levels, and salaries. Current results include Python as the most requested skill with 3 postings, 4 hybrid and 2 remote opportunities, 5 junior and 1 senior roles, an average minimum salary of \$105,500, and an average maximum salary of \$144,333.33.

### Final Big Data Job Search Architecture



Current MVP: local JSON and in-memory processing | Future scale: APIs, Spark, streaming, embeddings, vector database

## Job Search Dataset Analytics



## 8. Final Application

Job data -> ingestion -> cleaning -> analytics and BM25 index  
-> candidate selection -> retrieval -> weighted matching  
-> ranked recommendations -> skill gaps and explanations  
-> application draft and roadmap -> Streamlit interface

app.py provides candidate selection, weighted or BM25-hybrid retrieval, score thresholds, ranked opportunities, score charts, matched and missing skills, recommendations, and analytics. The application can be started with:

```
pip install -r requirements.txt
streamlit run app.py
```

## 9. Results and Evaluation

| Seeker        | Top recommendation               | Score  |
|---------------|----------------------------------|--------|
| Alice Johnson | Senior Machine Learning Engineer | 95.33% |
| Bob Smith     | Full Stack JavaScript Developer  | 96.25% |
| Carol Davis   | Product Manager - Consumer Apps  | 96.25% |
| Vince Nguyen  | Cybersecurity SOC Analyst        | 95.25% |

The weighted matcher averaged 2.7488 ms per seeker and reached a maximum measured time of 3.7977 ms on the local dataset. The current test suite contains 17 passing tests, including personal RAG chunking, hybrid evidence retrieval, and hard-gating checks.

| Method                           | What it does                                                                   | Top-1 results for Alice, Bob, Carol, Vince | Evidence and limitation                                                       |
|----------------------------------|--------------------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------|
| Keyword + weighted matcher       | Scores all jobs with keyword, skill, preference, salary, and experience rules. | JOB001, JOB002, JOB003, JOB006             | 2.7488 ms average; exact wording can miss synonyms.                           |
| BM25 retrieval                   | Uses term frequency, inverse document frequency, and length normalization.     | JOB001, JOB002, JOB003, JOB006             | Scores saved in `hybrid_retrieval_report.json`; no semantic embeddings.       |
| Hybrid BM25 + weighted reranking | Retrieves lexical candidates, then applies candidate-specific fit scoring.     | JOB001, JOB002, JOB003, JOB006             | Scores saved in `hybrid_retrieval_report.json`; no external relevance labels. |

All three methods identify the expected role at rank one for this curated sample. This demonstrates functionality, not statistical superiority.

## 10. Human vs. AI vs. Human-AI

| Aspect                | Human Design                                        | AI Design                                                  | Human-AI Co-Design                                                        |
|-----------------------|-----------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------|
| Problem understanding | Defined users, inputs, outputs, and seven criteria. | Converted requirements into classes, matcher, and tools.   | Evaluated whether generated scores represented realistic fit.             |
| Big-data architecture | Designed the conceptual flow.                       | Proposed modular local JSON/Python architecture.           | Added ingestion, analytics, retrieval, reranking, and Streamlit.          |
| Data processing       | Structured fields and keywords.                     | Generated loading, dataclasses, tokenization, and aliases. | Added normalization, defaults, duplicate checks, and quality reports.     |
| Analytics             | Planned weighted comparison.                        | Implemented scoring and explanations.                      | Added statistics, charts, evaluation, and timing.                         |
| Matching/retrieval    | Transparent weighted matching.                      | Added hierarchy, bonuses, aliases, and agent tools.        | Preserved explainability and added BM25 reranking.                        |
| Scalability           | Identified big-data concerns.                       | Delivered a local MVP.                                     | Documented scale boundaries and future architecture.                      |
| Code quality          | Supplied rules and priorities.                      | Generated modules quickly.                                 | Corrected defects and added regression tests.                             |
| Final quality         | Clear but limited concept.                          | Working agent MVP.                                         | Reproducible application with retrieval, analytics, interface, and tests. |

AI was most useful for implementation speed and structure. Human judgment was essential for debugging the score logic, preserving meaningful acronyms, validating results, and identifying unsupported scalability claims.

## 11. Reproducibility and Discussion

The repository contains the README, requirements, source code, JSON data, tests, results, screenshots, and the final Markdown report. Reproduce the project with:

```
python -m unittest discover -s tests -p "test_*.py" -v
python main.py --analytics --hybrid
streamlit run app.py
```

The current limitations are the small author-created dataset, local JSON storage, deterministic hashed dense vectors rather than a pretrained embedding model, no live updates, no distributed processing, no external relevance labels, and no formal fairness evaluation. Spark/PySpark,

streaming ingestion, live APIs, cloud storage, pretrained embeddings such as text-embedding-004, semantic vector databases such as Chroma or FAISS, and external LLM synthesis are future extensions only. They are not implemented features of this submission.

The generated results/personal\_rag\_evidence.json artifact records the top retrieved career-evidence chunks for each seeker's top recommendation, including chunk IDs and sparse, dense, and hybrid scores.

Human Design taught me to define the problem and assumptions first. AI contributed rapid implementation and agent structure. Human-AI co-design made the system more reliable by testing AI output, correcting its assumptions, and extending the application into an explainable, testable, and visual big-data prototype.